OpenZeppelin | security

OpenZeppelin Confidential Contracts BatcherConfidential and Diff Audit

ZAMA

March 20, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

BatcherConfidential 5

Changes 6

Security Model and Trust Assumptions 7

Privileged Roles 8

Medium Severity 9

M-01 dispatchBatchCallback Lacks Reentrancy Protection Despite External Calls in _executeRoute 9

Low Severity 10

L-01 A Dispatched Batch Can Be Permanently Bricked With No In-Protocol Recovery 10

L-02 Undocumented Invariant in Partial Route Execution May Cause Cross-Batch Fund Loss 11

L-03 Zero-Deposit Batch Dispatch Creates a Permanently Unresolvable State 12

Notes & Additional Information 13

N-01 Toolchain Mismatch Between Hardhat and Foundry 13

N-02 IERC7984ERC20Wrapper Interface Is Incomplete and Contains a Misplaced ACL Note 13

N-03 Missing and Misleading Documentation 14

Conclusion 16

Appendix 17

Issue Classification 17

Summary

Type	Library	Total Issues	7 (7 resolved)
Timeline	From 2026-02-24 To 2026-03-05	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	1 (1 resolved)
		Low Severity Issues	3 (3 resolved)
		Notes & Additional Information	3 (3 resolved)

Scope

OpenZeppelin audited the OpenZeppelin/openzeppelin-confidential-contracts repository at commit 9ceb3f0. In addition, a diff audit comparing commit 14f20de with commit 136840e was also performed.

In scope were the following files:

```
contracts/
├── governance/
│   └── utils/
│       └── VotesConfidential.sol (diff)
├── interfaces/
│   ├── IERC7984ERC20Wrapper.sol (diff)
│   ├── IERC7984Receiver.sol (diff)
│   └── IERC7984Rwa.sol (diff)
├── token/
│   └── ERC7984/
│       └── extensions/
│           ├── ERC7984ERC20Wrapper.sol (diff)
│           ├── ERC7984Freezable.sol (diff)
│           ├── ERC7984Restricted.sol (diff)
│           ├── ERC7984Rwa.sol (diff)
│           └── ERC7984Votes.sol (diff)
└── utils/
    ├── BatcherConfidential.sol (full)
    ├── FHESafeMath.sol (diff)
    └── HandleAccessManager.sol (diff)
```

***Update**: The fixes for the findings highlighted in this report have all been merged at commit c027469.*

System Overview

OpenZeppelin Confidential Contracts is a Solidity library of confidential token primitives built with Zama's FHEVM library. It extends the familiar OpenZeppelin Contracts architecture to support encrypted token balances and transfers, where sensitive financial data is processed as ciphertexts that are never exposed on-chain in plaintext without an explicit, gated decryption step. One essential contract of the library is the ERC-7984 token standard, a confidential analog to ERC-20, with a set of extensions and utilities that compose on top of it.

`BatcherConfidential`

`BatcherConfidential` is an abstract contract that aggregates confidential deposits of a `fromToken` from multiple users and routes them through a non-confidential swap to produce a `toToken`, where both tokens are `ERC7984ERC20Wrapper` instances. The batching model amortizes the cost of a swap across many users and provides a degree of confidentiality by obscuring individual trade sizes within the aggregate.

A batch progresses through the following lifecycle:

1. **Pending**: The batch is open. Users deposit encrypted `fromToken` balances via the `IERC7984Receiver` callback and can exit with their original deposit at any time.
2. **Dispatched**: The batch is closed. The encrypted total deposit is submitted for decryption through the two-step unwrap mechanism of `ERC7984ERC20Wrapper`, yielding a plaintext amount of underlying tokens held by the batcher.
3. **Route execution**: Once the unwrapping is finalized, `_executeRoute` is called with the plaintext unwrap amount. Implementations perform the swap here. A `Partial` outcome allows multi-step routes across repeated calls. A `Cancel` outcome rewraps the underlying tokens, so depositors can reclaim their original balance.
4. **Finalized**: On a `Complete` outcome, the received underlying `toToken` is wrapped back into the confidential token and an exchange rate is computed. Depositors can then claim their proportional share of `toToken`.

Concrete deployments must implement `_executeRoute` to define the swap logic and `routeDescription` to provide a human-readable description. The base contract places no access controls on dispatch or callback entry points. Thus, restrictions are left to the implementing contract.

Changes

The following summarizes modifications to the in-scope files:

- **`IERC7984ERC20Wrapper`**: A new interface for `ERC7984ERC20Wrapper` defining `wrap`, `unwrap` (with input proof variant), and `underlying`.
- **`ERC7984ERC20Wrapper`**: Implements `IERC7984ERC20Wrapper` and `IERC165`. The `wrap` and `unwrap` functions now return the encrypted amount sent. The `underlying` function now returns `address` instead of `IERC20`. A `supportsInterface` override has been added for ERC-165 detection. A public `unwrapRequester` `view` function exposes the pending unwrap request mapping. The `wrap` function now calls `FHE.allowTransient` to grant the caller transient access to the returned ciphertext. The `_unwrap` `internal` function now returns the unwrap ciphertext handle. The name of the `burntAmount` parameter has been updated to `unwrapAmount` throughout the codebase. A developer warning about the ciphertext uniqueness assumption in the unwrap request mapping has been added inline.
- **`IERC7984Receiver`**: The `onConfidentialTransferReceived` documentation has been updated to clarify that returning `false` causes the token contract to attempt a refund. It also added a warning against manually refunding while also returning `false` to prevent double refunds.
- **`IERC7984Rwa`**: `isUserAllowed` has been renamed to `canTransact`.
- **`ERC7984Restricted`**: `isUserAllowed` has been renamed to `canTransact` throughout the codebase.
- **`ERC7984Rwa`**: `isUserAllowed` has been renamed to `canTransact`. The swapped inline comments on the two `forceConfidentialTransferFrom` selector values in `_isForceTransfer` have been fixed.
- **`ERC7984Freezable`**: Documentation has been updated by removing a reference to an authorized account role.
- **`ERC7984Votes`**: A typo in the NatSpec has been fixed.
- **`HandleAccessManager`**: The pragma has been bumped to `^0.8.26`. In addition, the `_validateHandleAllowance` function documentation has been changed from reverting to returning a `bool`. A dedicated `HandleAccessManagerNotAllowed` error has been added, and `getHandleAllowance` now uses a `require` check against the return value.
- **`VotesConfidential`**: The pragma has been bumped to `^0.8.26`.
- **`FHESafeMath`**: A grammar error in the NatSpec has been fixed.

Security Model and Trust Assumptions

The security of this system is grounded in the correctness guarantees of the underlying FHEVM and its associated access control layer. Encrypted values are processed by co-processors that enforce well-formed operations, and decryptions require valid proofs issued by the FHEVM oracle. The on-chain contracts validate these proofs before acting on any plaintext value. Beyond the cryptographic layer, the system inherits the role-based access assumptions common to the OpenZeppelin Contracts ecosystem.

Critical trust assumptions include the following:

- **FHEVM oracle integrity**: The decryption oracle is trusted to produce correct and timely cleartext values with valid proofs. A compromised or unavailable oracle permanently blocks all unwrapping and batch finalization flows.
- **Ciphertext uniqueness**: `ERC7984ERC20Wrapper` stores pending unwrap requests in a mapping keyed by ciphertext handle, assuming that ciphertexts produced by `_burn` are unique within the contract. This holds under the current FHEVM implementation but is not a universal FHE guarantee.
- **Route implementation correctness**: The security of a deployed batcher entirely depends on the correctness of `_executeRoute`. A buggy or malicious implementation can cause permanent loss of user deposits.
- **Route termination**: If `_executeRoute` returns `Partial` indefinitely without resolving to `Complete` or `Cancel`, user deposits will be permanently locked. The base contract provides no timeout or forced-cancel mechanism.
- **No confidentiality guarantees by default**: Individual deposit sizes and batch compositions can be inferred from events and public state. Confidentiality properties are application-specific and require additional restrictions imposed by the implementing contract.
- **Underlying ERC-20 token assumptions**: The wrapper does not support fee-on-transfer or deflationary tokens. Deploying with such tokens will result in the contract holding less underlying collateral than what the encrypted supply implies.
- **`HandleAccessManager` authorization**: The `_validateHandleAllowance` hook is unimplemented by design. The security of handle access grants depends entirely on the correctness of this override in each consuming contract.

Privileged Roles

The `BatcherConfidential` contract does not define any privileged roles. The `dispatchBatch` and `dispatchBatchCallback` entry points are permissionless, and any access restrictions are left to the implementing contract.

`ERC7984Rwa` defines two roles via OpenZeppelin `AccessControl`. The `DEFAULT_ADMIN_ROLE` has been granted to the address passed to the constructor and is responsible for granting and revoking the `AGENT_ROLE`. Accounts holding the `AGENT_ROLE` have broad operational capabilities: minting and burning tokens, pausing and unpausing the contract, blocking and unblocking accounts, setting confidential frozen balances, and executing force transfers that bypass pause and sender restriction checks. The security of the system depends on the integrity of these role holders.

Medium Severity

M-01 `dispatchBatchCallback` Lacks Reentrancy Protection Despite External Calls in `_executeRoute`

Unlike `claim` and `quit`, `dispatchBatchCallback` does not have a reentrancy guard. All state-finalizing writes, setting `exchangeRate` on the Complete path and `canceled` on the Cancel path, occur after the invocation of `_executeRoute`, which will perform an external call. A child contract that routes through an untrusted contract could expose the batcher to reentrancy at this point.

If a contract invoked by `_executeRoute` re-enters `dispatchBatchCallback` for the same batch, the batch is still in `Dispatched` state at that point, so the state check passes. The reentrant call runs `_executeRoute` a second time and, if it returns `Complete`, sets `exchangeRate` and emits `BatchFinalized`. When control returns to the outer call, `_executeRoute` completes and the outer call overwrites `exchangeRate` with a value computed on a now-depleted batcher balance, potentially setting it to zero or triggering the `InvalidExchangeRate` revert. This would leave the batch in a finalized-but-broken state with the exchange rate already written by the inner call.

Consider adding a `nonReentrant` modifier to `dispatchBatchCallback`, consistent with the reentrancy protection already applied to `claim` and `quit`.

Update: *Resolved at commit 683010e. The OpenZeppelin Zama development team stated:*

> *Reentrancy protection has been added.*

Low Severity

L-01 A Dispatched Batch Can Be Permanently Bricked With No In-Protocol Recovery

Once a batch is dispatched, `dispatchBatchCallback` must succeed for deposited funds to be recoverable. `ERC7984ERC20Wrapper` enforces a hard capacity ceiling on the confidential total supply: any `wrap` call that would push it past `type(uint64).max` reverts with `ERC7984TotalSupplyOverflow` (or `SafeCastOverflowedUintDowncast` for a single-mint overflow). When `dispatchBatchCallback` hits this ceiling, or any other revert condition is met, the batch gets stuck in the `Dispatched` state with no in-protocol transition available.

Several conditions cause `dispatchBatchCallback` to revert irrecoverably:

1. An attacker fills the `toToken` wrapper to capacity before `dispatchBatchCallback` is called, causing the wrap call to revert.
2. The route produces fewer underlying tokens than `toToken().rate()`, so the wrapped output rounds to zero. No subsequent call can finalize the batch.
3. The exchange rate rounds to zero when `wrappedAmount * 10^6 < unwrapAmountCleartext`, causing the exchange rate require to fire.
4. An attacker fills the `fromToken` wrapper to capacity when the route returns `ExecuteOutcome.Cancel`, causing the re-wrap on the cancel path to revert.

In all four cases, the batch is stuck in the `Dispatched` state with no transition available. At the moment of dispatch, both token amounts sit in unwrapped form. So, an attacker can fill both wrappers simultaneously, blocking finalization and cancellation in a single coordinated action.

Consider a confidentiality-breaking emergency recovery path that does not rely on re-wrapping either token. Since `finalizeUnwrap` is permissionless, anyone can call it directly on the `fromToken` wrapper to land the unwrapped underlying in the batcher contract. From there, an emergency withdrawal function can let each depositor prove their deposit cleartext by supplying the plaintext value and a `FHE.checkSignatures` proof over their `_batches[batchId].deposits[msg.sender]` handle. A corresponding proof over the `totalDeposits(batchId)` handle is needed to verify the denominator used in the proportional share calculation. With both values verified, the function calculates the depositor's share of the underlying balance and transfers it directly, bypassing both wrappers entirely. This

breaks deposit confidentiality for participants who invoke the emergency path, which is an acceptable trade-off when finalization and all re-wrap-based recovery paths are permanently blocked.

Alternatively, consider clearly documenting the fact that wrappers need to be carefully chosen. Furthermore, implementers should monitor the capacity of both wrappers and users should desist from joining a batch when the capacity is close to being reached.

Update: *Resolved at commit 5d1ed4a and at commit 32bf966. The OpenZeppelin Zama development team stated:*

> *An explicit warning of this situation was added.*

L-02 Undocumented Invariant in `Partial` Route Execution May Cause Cross-Batch Fund Loss

`BatcherConfidential` intentionally uses a full `balanceOf(toToken.underlying)` snapshot in the `Complete` branch to collect dust left over from prior wrap operations. This design is correct in isolation, but it introduces a silent invariant that `_executeRoute` implementations must never deposit any `toToken` underlying to the batcher during a `Partial` step. This invariant is not documented anywhere in the codebase.

If a `Partial` implementation violates this (for example, a multi-hop route that receives partial `toToken` output before the final step settles) and a second batch is concurrently in `Dispatched` state, the consequences are severe. The concurrent batch's `Complete` callback sweeps the entire shared balance including the other batch's prematurely deposited output, inflating its exchange rate. The affected batch then has insufficient output to produce a valid exchange rate on its next call, reverting permanently with `InvalidExchangeRate` and locking all depositors' funds in the `Dispatched` state indefinitely.

Consider adding explicit NatSpec to `_executeRoute` stating that `Partial` steps must not transfer any `toToken` underlying to the batcher, and that all `toToken` output must be deferred until the step that returns `Complete`. Given the severity of a violation, also consider adding a note to the `Partial` enum value itself and to the `dispatchBatchCallback` NatSpec.

Update: *Resolved at commit f5a6ecd. The OpenZeppelin Zama development team stated:*

> *Additional documents have been added.*

L-03 Zero-Deposit Batch Dispatch Creates a Permanently Unresolvable State

The `dispatchBatch` function is permissionless and does not place a guard on the deposit state of the current batch before advancing it. It can therefore be called on a batch that decrypts to zero total deposits, either because no user has joined or because the only joins carried a zero amount. In both cases, `_calculateUnwrapAmount` produces a valid non-zero FHE handle encoding encrypted zero, the batch ID is incremented, and `BatchDispatched` is emitted. The batch is correctly classified as `Dispatched` by `batchState` but has no path to resolution.

When `dispatchBatchCallback` is subsequently called with `unwrapAmountCleartext == 0`, the call passes state validation and finalizes the unwrap. It then reaches the exchange rate calculation, which divides by `unwrapAmountCleartext`, reverting with a division by zero. The batch is permanently stuck in `Dispatched` with no transition to `Finalized` or `Canceled` possible, violating the lifecycle invariant stipulating that every dispatched batch must eventually reach a terminal state.

Consider two complementary fixes. First, in `dispatchBatch`, add a guard requiring that `totalDeposits` is initialized before proceeding. This is a weak guard: it rejects batches where no user has joined at all, but a user joining with a zero amount leaves `totalDeposits` initialized while still decrypting to zero, bypassing the check. Second, and more robustly, handle `unwrapAmountCleartext == 0` explicitly in `dispatchBatchCallback` immediately after the `try`/`catch` block, at which point `unwrapAmountCleartext` has already been validated against the cipher-text in both branches. Set `_batches[batchId].canceled = true`, emit `BatchCanceled`, and return early. This covers both trigger scenarios and ensures that every dispatched batch can reach a terminal state.

Update: *Resolved at commit eb23ca3. The OpenZeppelin Zama development team stated:*

> *The batch shortcuts to canceled if the amount is revealed to be 0 upon batch callback.*

Notes & Additional Information

N-01 Toolchain Mismatch Between Hardhat and Foundry

The codebase currently specifies different Solidity compiler versions and EVM targets in `foundry.toml` and `hardhat.config.ts`. Using different compiler versions and EVM targets across toolchains can result in divergent bytecode output between Foundry and Hardhat, even when compiling the same source code. Differences between Solidity 0.8.27 and 0.8.29 may include bug fixes, regressions, or backend code generation changes that subtly affect contract behavior. Additionally, targeting different EVM versions (prague vs cancun) can influence opcode availability and code generation assumptions. Together, these discrepancies increase the likelihood of inconsistent test results depending on the framework used and introduce a risk that deployed artifacts are compiled under different assumptions than those validated during testing.

Consider unifying the Solidity compiler version and EVM target across both configurations to ensure deterministic builds and consistent testing.

Update: *Resolved in pull request #315 at commit 47dd453. The OpenZeppelin Zama development team stated:*

> *Foundry and hardhat solc versions were synchronized.*

N-02 `IERC7984ERC20Wrapper` Interface Is Incomplete and Contains a Misplaced ACL Note

`IERC7984ERC20Wrapper` is a new interface introduced to represent the public API of `ERC7984ERC20Wrapper`. Two issues reduce its accuracy:

- The ACL note on `unwrap` states "The caller must already be approved by ACL for the given `amount`". This note is attached to the `externalEuint64` overload, for which it does not apply: the caller provides an `inputProof` that grants ACL access automatically via `FHE.fromExternal`. The note belongs to the `euint64` overload, which requires pre-existing ACL approval but is not declared in the interface.

- `finalizeUnwrap` and `unwrapRequester` are absent from the interface. The implementation's `unwrap` NatSpec explicitly states that the request must be finalized by calling `finalizeUnwrap`, yet a caller holding only an `IERC7984ERC20Wrapper` reference has no interface-level way to look up pending requests or complete the unwrap flow.

Consider moving the ACL note to the `euint64` overload (or adding that overload to the interface), and adding `finalizeUnwrap` and `unwrapRequester` to `IERC7984ERC20Wrapper`. Doing so will ensure that the interface fully covers the unwrap lifecycle.

Update: *Resolved in pull request #323 at commit 59f4f51. The `unwrapRequester` and `unwrap` (with `euint64` type) functions were not added to the interface because they are considered non-core functionality and obsolete, respectively. The OpenZeppelin Zama development team stated:*

> *We decided to add more functions to `IERC7984ERC20Wrapper,` which makes it fully useable independently. This may be the foundation for a new ERC. Thank you for this feedback.*

N-03 Missing and Misleading Documentation

Throughout the codebase, the following instances of missing and/or misleading documentation were identified:

- `BatchCanceled` and `BatchFinalized` reference `{_cancel}` and `{_setExchangeRate}`, respectively. However, neither function exists. Both events are emitted inside `dispatchBatchCallback`.
- The `InvalidExchangeRate` error description states that `exchangeRate * totalDeposits / 10 ** exchangeRateDecimals` must fit in `uint64`. This describes the per-user claim calculation, not the conditions that trigger the error. The error fires when the computed exchange rate rounds to zero or when `wrappedAmount` exceeds `type(uint64).max`.
- `dispatchBatchCallback` does not document the fact that it can be called repeatedly when `_executeRoute` returns `Partial`.
- The `_join` parameter is named `to` despite receiving the depositor address, which is named `from` at every call site. The name conflicts with the `to = recipient` convention used throughout the rest of the contract.

- `ERC7984ERC20Wrapper.onTransferReceived` contains a typo: "See {wrap} **from** more details on wrapping tokens" should read "See {wrap} **for** more details on wrapping tokens".
- The `_unwrap` inline comment states "cipher-texts are unique--this holds here but is not always true" without explaining why it holds. The comment should note that uniqueness is guaranteed here because in `_burn` and `_update` an always changing `totalSupply` produces a distinct ciphertext handle each time.
- The `getHandleAllowance` NatSpec states "This function call is gated by `msg.sender`", but the function itself performs no `msg.sender` check. The caller validation is entirely delegated to the abstract `_validateHandleAllowance`. The NatSpec should instead direct implementors to enforce `msg.sender` authorization inside `_validateHandleAllowance`.
- The `BatcherConfidential` contract-level NatSpec comment "developers should consider restricting certain functions to increase confidentiality" is vague and could be more explicit, for instance, suggesting to guard `dispatchBatch` with access control, or introducing a counter of users who have already joined a batch.
- While `toToken` underlying dust is handled via `balanceOf` accounting, additional dust is introduced during claiming due to truncation from the `uint128(10) ** exchangeRateDecimals()` division. This may leave residual unclaimable amounts and can cause very small deposits to yield zero output tokens. However, this behavior is not documented anywhere.

Consider updating the documentation to correct the identified inaccuracies and omissions.

Update: *Resolved in pull request #316 at commit 4323555, 417c7dd and at commit c027469. The OpenZeppelin Zama development team stated:*

> *Most findings here were fixed. Some were not for the following reasons:*
>
> *- `_join` parameter name: we don't currently support joining on behalf of a different account yet, but that is essentially what the naming allows for here. Elsewhere, we understand who the funds are coming from, here we decide who the future claim is going to.*
>
> *- We intentionally left the details on how the handle uniqueness holds vague. This is an anti-pattern that we don't encourage in general. Users should dig deeply to be able to understand the nuance of it.*
>
> *- We didn't add specific recommendations as to how to make batcher actually confidential, as it is such an application-specific solution. We don't want to point users in the wrong direction and be at fault if they leak data.*

Conclusion

The audited codebase introduces `BatcherConfidential`, an abstract contract that aggregates encrypted deposits from multiple users and routes them through a non-confidential swap. In addition to the new batching component, the audit covered targeted modifications to in-scope files, including interface additions, consistent renaming, documentation improvements, minor behavioral adjustments, ERC-165 support, explicit custom errors, and pragma bumps. Overall, the diff reflects incremental refinements and clarity improvements rather than architectural changes.

The audit findings were limited to low-severity issues and notes, covering topics such as documentation clarity, missing NatSpec coverage, implicit trust assumptions, and the absence of protective mechanisms against edge-case lifecycle scenarios. These observations are consistent with a mature codebase that builds carefully on established OpenZeppelin patterns. The codebase is well-structured, though the documentation could benefit from more explicit guidance on the responsibilities and security considerations left to concrete deployments of `BatcherConfidential`.

The OpenZeppelin Zama development team is greatly appreciated for their responsiveness and collaborative approach throughout the audit.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.